Context-Free Grammars

Generating Recursive Structure

Sheikh Azizul Hakim

CSE 211: Theory of Computation

Department of Computer Science and Engineering, BUET

Where We Are Going

Finite automata recognize patterns with finite memory.

Context-free grammars generate strings with recursive, nested, or matched structure.

Guiding question

How can a finite set of rules generate infinitely many structured strings?

Learning Goals

By the end of this lecture, you should be able to:

1. define a context-free grammar formally;
2. perform derivations using grammar rules;
3. distinguish derivations, leftmost derivations, rightmost derivations, and parse trees;
4. design CFGs for diverse recursive languages;
5. identify and repair common ambiguity in grammars;
6. simplify grammars by removing useless, nullable, and unit productions;
7. convert a grammar to Chomsky Normal Form;
8. run the CYK membership algorithm on a small example.

Why Grammars?

From Recognition to Generation

A finite automaton asks:

Is this string accepted?

A grammar asks:

How could this string be built?

Recognizer view

Read input from left to right and decide yes/no.

Generator view

Start from a symbol and repeatedly expand it until only terminals remain.

The generator view is natural for syntax: programs, expressions, nested calls, trees, brackets, and protocols.

Recursive Objects Are Everywhere

Nested brackets

`([]{()})`

A valid object may contain smaller valid objects.

Function calls

`encrypt(hash(msg))`

An argument may itself be a call.

Mirrored packets

`101#101`

The second part checks the first part in reverse.

All three examples require a rule like:

> a large valid object contains smaller valid objects.

A Grammar Before the Definition

Consider properly nested parentheses:

$$\varepsilon, \quad (), \quad ()(), \quad (()), \quad (()()), \quad ((())())$$

A compact set of construction rules is:

$$S \to SS \mid (S) \mid \varepsilon.$$

$S \to \varepsilon$

The empty string is balanced.

$S \to (S)$

Wrapping a balanced string keeps it balanced.

$S \to SS$

Concatenating two balanced strings keeps it balanced.

Worked Derivation: Generating $(()())$

Using

$$S \to SS \mid (S) \mid \varepsilon,$$

we can derive:

$$\begin{aligned} S &\Rightarrow (S) \\ &\Rightarrow (SS) \\ &\Rightarrow ((S)S) \\ &\Rightarrow (()S) \\ &\Rightarrow (()(S)) \\ &\Rightarrow (()()). \end{aligned}$$

Observation

A derivation is a construction history. The same final string may sometimes have more than one construction history.

What a Grammar Is Good At

A grammar is good when strings can be built by repeatedly applying a few local construction principles.

Key intuition

A CFG has only finitely many rules, but recursion lets those rules be reused arbitrarily many times.

Formal Definition

Formal Definition of a CFG

A context-free grammar is a 4-tuple

$$G = (V, \Sigma, R, S),$$

where:

- V is a finite set of variables or nonterminals;
- Σ is a finite set of terminals, disjoint from V;
- R is a finite set of productions;
- $S \in V$ is the start variable.

Each production has the form

$$A \to \alpha,$$

where $A \in V$ and $\alpha \in (V \cup \Sigma)^*$.

Variables Versus Terminals

Variables

Variables are placeholders for structured pieces not yet fully expanded.

$$S, \quad E, \quad T, \quad Factor, \quad Stmt$$

Terminals

Terminals are the actual symbols that appear in the final string.

$$0, 1, +, *, (,), \texttt{if}, \texttt{else}$$

A derivation ends successfully only when no variables remain.

$$S \overset{*}{\Rightarrow} w, \qquad w \in \Sigma^*.$$

Example Grammar as a 4-Tuple

For mixed brackets, let

$$S \to SS \mid (S) \mid [S] \mid \{S\} \mid \varepsilon.$$

Then

$$G_{br} = (V, \Sigma, R, S),$$

where

$$V = \{S\},$$
$$\Sigma = \{(,), [,], \{, \}\},$$

and R is the set of five productions above.

Language generated

$$L(G_{br}) = \{w \mid w \text{ is a properly nested mixed-bracket string}\}.$$

Reading Production Rules

A rule

$$A \to \alpha$$

means:

Whenever the current sentential form contains A, we may replace that occurrence by α.

For example, if

$$S \to 0S1 \mid \#,$$

then

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 00\#11.$$

Context-free means

The rule for replacing A does not depend on the symbols around that occurrence of A.

Sentential Forms and Derivations

A sentential form is any string over variables and terminals that can appear during a derivation.

For the grammar

$$S \to 0S1 \mid \#,$$

we have:

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 00\#11.$$

Step	Sentential form
0	S
1	$0S1$
2	$00S11$
3	$00\#11$

The final line is a terminal string.

The Language of a Grammar

For a CFG $G = (V, \Sigma, R, S)$, the language of G is

$$L(G) = \{w \in \Sigma^* \mid S \overset{*}{\Rightarrow} w\}.$$

A language L is context-free if there exists some CFG G such that

$$L = L(G).$$

Important distinction

A grammar is a description. A language is the set of strings described by the grammar.

Derivations

A Tiny Grammar for Nested Commands

Let us generate command strings such as

$$\texttt{encrypt(sign(data))}.$$

Use the grammar:

$$S \to D \mid F(S),$$
$$F \to \texttt{hash} \mid \texttt{sign} \mid \texttt{encrypt},$$
$$D \to \texttt{data} \mid \texttt{msg} \mid \texttt{key}.$$

Interpretation

A command is either raw data, or a function applied to a smaller command.

Worked Derivation: `encrypt(sign(data))`

$$
\begin{aligned}
S &\Rightarrow F(S) \\
&\Rightarrow \texttt{encrypt}(S) \\
&\Rightarrow \texttt{encrypt}(F(S)) \\
&\Rightarrow \texttt{encrypt}(\texttt{sign}(S)) \\
&\Rightarrow \texttt{encrypt}(\texttt{sign}(D)) \\
&\Rightarrow \texttt{encrypt}(\texttt{sign}(\texttt{data})).
\end{aligned}
$$

What recursion did

The outer command was generated first, but it forced the inside to be another valid command.

Leftmost and Rightmost Derivations

A sentential form may contain several variables.

Leftmost derivation

At each step, replace the leftmost variable.

$$\Rightarrow_{\mathrm{lm}}$$

Rightmost derivation

At each step, replace the rightmost variable.

$$\Rightarrow_{\mathrm{rm}}$$

Important

Leftmost and rightmost derivations control the order of expansion. They do not necessarily change the final parse tree.

Worked Leftmost Derivation

Use

$$E \to E + T \mid T, \quad T \to T * F \mid F, \quad F \to (E) \mid a.$$

A leftmost derivation of $a + a * a$:

$$\begin{aligned} E &\Rightarrow_{\text{lm}} E + T \\ &\Rightarrow_{\text{lm}} T + T \\ &\Rightarrow_{\text{lm}} F + T \\ &\Rightarrow_{\text{lm}} a + T \\ &\Rightarrow_{\text{lm}} a + T * F \\ &\Rightarrow_{\text{lm}} a + F * F \\ &\Rightarrow_{\text{lm}} a + a * F \\ &\Rightarrow_{\text{lm}} a + a * a. \end{aligned}$$

Worked Rightmost Derivation

Using the same grammar:

$$
\begin{aligned}
E &\Rightarrow_{\mathrm{rm}} E + T \\
&\Rightarrow_{\mathrm{rm}} E + T * F \\
&\Rightarrow_{\mathrm{rm}} E + T * a \\
&\Rightarrow_{\mathrm{rm}} E + F * a \\
&\Rightarrow_{\mathrm{rm}} E + a * a \\
&\Rightarrow_{\mathrm{rm}} T + a * a \\
&\Rightarrow_{\mathrm{rm}} F + a * a \\
&\Rightarrow_{\mathrm{rm}} a + a * a.
\end{aligned}
$$

Same string, same intended parse

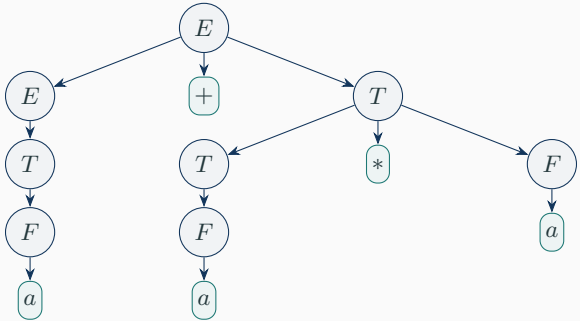
The expansion order changed, but the multiplication still binds inside the final T.

Derivations Are Linear; Structure Is a Tree

A derivation is written as a sequence.

$$E \Rightarrow E + T \Rightarrow T + T \Rightarrow F + T \Rightarrow a + T \Rightarrow \cdots$$

But the object being built is hierarchical.



Parse Trees

Parse Trees: Formal Conditions

For a CFG $G = (V, \Sigma, R, S)$, a parse tree satisfies:

1. Each internal node is labeled by a variable.
2. Each leaf is labeled by a variable, terminal, or ε.
3. If an internal node labeled A has children labeled

   $$X_1, X_2, \ldots, X_k,$$

   then

   $$A \to X_1 X_2 \cdots X_k$$

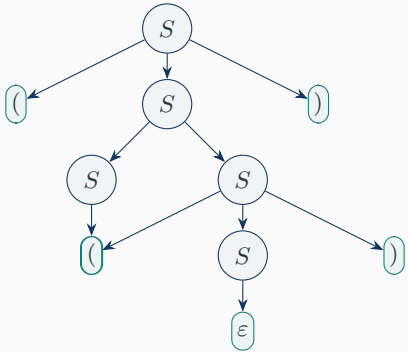
   is a production.

Special case

If a leaf is labeled ε, it is the only child of its parent.

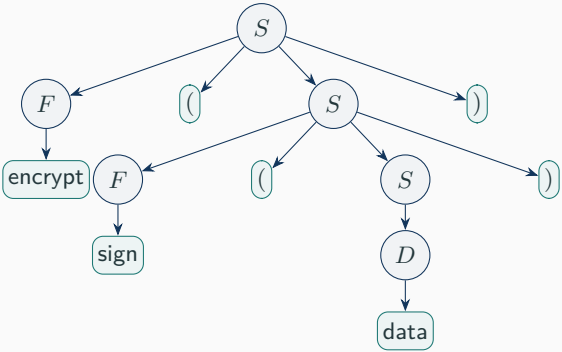
Yield of a Parse Tree

The yield of a parse tree is obtained by reading its leaves from left to right.



yield = (())

Parse Tree for a Nested Command



yield = `encrypt(sign(data))`

Derivations and Parse Trees Are Equivalent

For every variable A and terminal string w, the following are equivalent:

1. $A \overset{*}{\Rightarrow} w$.
2. $A \overset{*}{\Rightarrow}_{\mathrm{lm}} w$.
3. $A \overset{*}{\Rightarrow}_{\mathrm{rm}} w$.
4. There is a parse tree with root A and yield w.

The proofs are by induction on derivation length or tree height.

Recursive Inference: Bottom-Up Thinking

Parse trees can be read top-down or bottom-up.

Top-down

Start from S, expand variables, and try to generate the input.

Bottom-up

Start from small substrings and infer which variables could have generated them.

The CYK algorithm at the end of the lecture is a systematic bottom-up parser for grammars in CNF.

Designing CFGs: Worked Examples

CFG Design Checklist

When designing a grammar, ask:

1. What is the smallest valid string?
2. How can a larger valid string be built from smaller valid strings?
3. Is there a natural first/last pair to generate together?
4. Is the language a union of simpler languages?
5. Does part of the language happen to be regular?
6. Which variables correspond to meaningful syntactic categories?
7. Why can no invalid string be generated?

Example 1: Balanced Mixed Brackets

$$L_1 = \{w \mid w \text{ is a properly nested string over } (), [], \{\}\}.$$

Grammar

$$S \to SS \mid (S) \mid [S] \mid \{S\} \mid \varepsilon.$$

Construction

Wrap a balanced object, or concatenate two balanced objects.

Sample strings

ε, `()`, `([])`, `[]{()}`

Worked Derivation: `[]{()}`

$$S \to SS \mid (S) \mid [S] \mid \{S\} \mid \varepsilon.$$

$$\begin{aligned} S &\Rightarrow SS \\ &\Rightarrow [S]S \\ &\Rightarrow []S \\ &\Rightarrow []\{S\} \\ &\Rightarrow []\{(S)\} \\ &\Rightarrow []\{()\}. \end{aligned}$$

Rejection intuition

Strings like `([)]` cannot be generated because every opening bracket must be closed by the matching wrapper rule.

Why Example 1 Works

Soundness

Every rule preserves proper nesting:

- ε is balanced;
- $(S), [S], \{S\}$ wrap a balanced string with matching brackets;
- SS concatenates two balanced strings.

Completeness

Every nonempty balanced string is either:

- one bracket pair surrounding a balanced inside; or
- a concatenation of two shorter balanced strings.

Example 2: Equal Numbers of 0s and 1s in Any Order

$$L_2 = \{w \in \{0,1\}^* \mid \#_0(w) = \#_1(w)\}.$$

A grammar is:

$$S \to 0S1S \mid 1S0S \mid \varepsilon.$$

Why this is interesting

The matching symbols need not be adjacent or appear in two blocks.

$$\texttt{001011}, \quad \texttt{0101}, \quad \texttt{1010} \in L_2.$$

Not merely 0^n1^n

This language allows arbitrary interleavings as long as the two counts agree.

Worked Derivation: 0101

Using

$$S \to 0S1S \mid 1S0S \mid \varepsilon,$$

we derive:

$$\begin{aligned} S &\Rightarrow 0S1S \\ &\Rightarrow 01S \\ &\Rightarrow 010S1S \\ &\Rightarrow 0101S \\ &\Rightarrow 0101. \end{aligned}$$

Invariant

Each nonempty production introduces one 0 and one 1. Therefore every generated terminal string has equal counts.

Why the Grammar for Equal Counts Is Complete

Take a nonempty balanced string w. Suppose it starts with 0.

Scan from left to right until the first point where the number of 0's and 1's in the prefix becomes equal.

Then

$$w = 0x1y$$

where x and y are both balanced.

So the rule

$$S \to 0S1S$$

can generate it recursively.

Symmetric case

If w starts with 1, use $S \to 1S0S$.

Example 3: Mirrored Packets

$$L_3 = \{w\#w^R \mid w \in \{0,1\}^*\}.$$

Grammar

$$S \to 0S0 \mid 1S1 \mid \#.$$

Construction

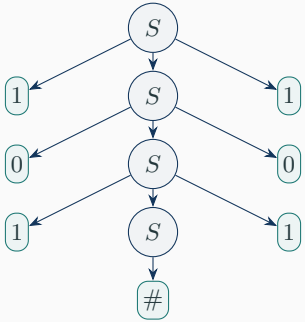
Choose the middle marker $\#$, then wrap equal symbols around it.

Examples

$$\#, \quad 0\#0, \quad 10\#01, \quad 101\#101.$$

Worked Derivation: $101\#101$

$$
\begin{aligned}
S &\Rightarrow 1S1 \\
&\Rightarrow 10S01 \\
&\Rightarrow 101S101 \\
&\Rightarrow 101\#101.
\end{aligned}
$$



Example 4: All Palindromes

$$L_4 = \{w \in \{0,1\}^* \mid w = w^R\}.$$

Grammar

$$P \to 0P0 \mid 1P1 \mid 0 \mid 1 \mid \varepsilon.$$

Even-length base

$$P \to \varepsilon$$

Examples: $\varepsilon, 00, 0110$.

Odd-length base

$$P \to 0 \mid 1$$

Examples: $0, 1, 10101$.

Example 5: Two Independent Matching Layers

$$L_5 = \{a^n b^m c^m d^n \mid n, m \geq 0\}.$$

Grammar

$$S \to aSd \mid T, \qquad T \to bTc \mid \varepsilon.$$

Idea

The outer recursion matches a's with d's. The inner recursion matches b's with c's.

$$\underbrace{a^n}_{S \text{ wraps}} \; \underbrace{b^m c^m}_{T \text{ generates}} \; \underbrace{d^n}_{S \text{ wraps}}$$

Worked Derivation: $a^2b^3c^3d^2$

$$
\begin{aligned}
S &\Rightarrow aSd \\
&\Rightarrow aaSdd \\
&\Rightarrow aaTdd \\
&\Rightarrow aabTcdd \\
&\Rightarrow aabbTccdd \\
&\Rightarrow aabbbTcccdd \\
&\Rightarrow aabbbcccdd.
\end{aligned}
$$

Design pattern

Use one variable for each independent obligation, and nest variables when one equality must sit inside another.

Example 6: One of Two Equalities

$$L_6 = \{a^i b^j c^k \mid i = j \text{ or } j = k\}.$$

Build the language as a union:

$$S \to S_1 \mid S_2.$$

Branch 1: verify $i = j$

$$S_1 \to AC, \quad A \to aAb \mid \varepsilon, \quad C \to cC \mid \varepsilon.$$

Branch 2: verify $j = k$

$$S_2 \to DB, \quad D \to aD \mid \varepsilon, \quad B \to bBc \mid \varepsilon.$$

Example 7: Marked Middle Palindromes

$$L_7 = \{wcw^R \mid w \in \{a, b\}^*\}.$$

Grammar

$$S \to aSa \mid bSb \mid c.$$

Why the marker helps

The symbol c explicitly marks the middle, so the grammar knows when to stop wrapping.

Worked derivation

$$S \Rightarrow aSa \Rightarrow abSba \Rightarrow abcba.$$

Example 8: Binary Trees in Prefix Form

Let x be a leaf and $n(T, T)$ be an internal node with two children.

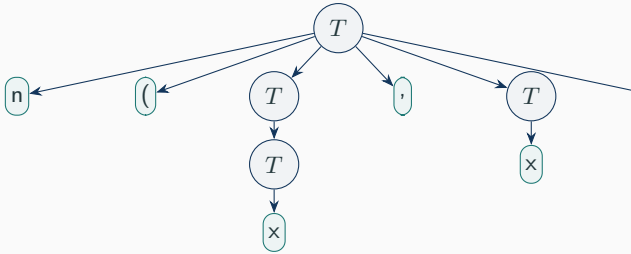
$$T \to x \mid n(T, T).$$

Generated strings

`x`

`n(x,x)`

`n(n(x,x),x)`



Example 9: A Tiny Query Language

Generate queries such as

`select(name,where(age>18)).`

$$
\begin{aligned}
Q &\to \texttt{select}(F, W),\\
F &\to \texttt{name} \mid \texttt{age} \mid \texttt{dept},\\
W &\to \texttt{all} \mid \texttt{where}(P),\\
P &\to F\ O\ V,\\
O &\to \texttt{>} \mid \texttt{=} \mid \texttt{<},\\
V &\to \texttt{18} \mid \texttt{30} \mid \texttt{cse}.
\end{aligned}
$$

Moral

Variables often represent syntactic categories, not just counting devices.

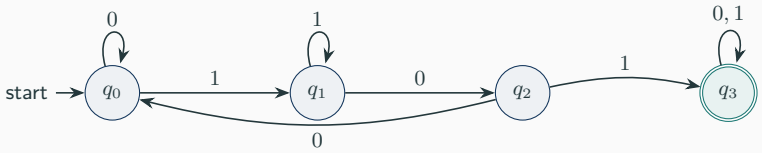
Worked Derivation: Query Language

$$
\begin{aligned}
Q &\Rightarrow \texttt{select}(F, W) \\
&\Rightarrow \texttt{select}(\texttt{name}, W) \\
&\Rightarrow \texttt{select}(\texttt{name}, \texttt{where}(P)) \\
&\Rightarrow \texttt{select}(\texttt{name}, \texttt{where}(FOV)) \\
&\Rightarrow \texttt{select}(\texttt{name}, \texttt{where}(\texttt{age}OV)) \\
&\Rightarrow \texttt{select}(\texttt{name}, \texttt{where}(\texttt{age} > V)) \\
&\Rightarrow \texttt{select}(\texttt{name}, \texttt{where}(\texttt{age} > 18)).
\end{aligned}
$$

Example 10: Converting a DFA to a CFG

Let

$$L_{10} = \{w \in \{0,1\}^* \mid w \text{ contains the substring } 101\}.$$



Make one variable R_i for each state q_i.

DFA-to-CFG Construction

For every transition $\delta(q_i, a) = q_j$, add

$$R_i \to aR_j.$$

For every accept state q_i, add

$$R_i \to \varepsilon.$$

The start variable is the variable corresponding to the DFA start state.

For the substring 101 DFA:

$$\begin{aligned} R_0 &\to 0R_0 \mid 1R_1, \\ R_1 &\to 1R_1 \mid 0R_2, \\ R_2 &\to 0R_0 \mid 1R_3, \\ R_3 &\to 0R_3 \mid 1R_3 \mid \varepsilon. \end{aligned}$$

Example 11: A Tempting Non-Example

Consider

$$L = \{ww \mid w \in \{0,1\}^*\}.$$

Students often try:

$$S \to 0S0 \mid 1S1 \mid \varepsilon.$$

But this generates

$$\{ww^R \mid w \in \{0,1\}^*\},$$

not $\{ww\}$.

Important intuition

Recursive wrapping remembers the first half in reverse order. This is exactly why stacks naturally recognize mirrors, not copies.

Design Patterns We Have Used

Pattern	Typical rule form
Wrapping	$S \to aSb$
Concatenation	$S \to SS$ or $S \to AB$
Optional structure	$A \to \alpha \mid \varepsilon$
Union of cases	$S \to S_1 \mid S_2$
Regular part	Convert DFA states into variables
Syntactic categories	One variable per category

Ambiguity

Ambiguous Grammars

A string w is derived ambiguously in a CFG G if it has two or more different parse trees.

Equivalently, it has:

- two or more different leftmost derivations; or
- two or more different rightmost derivations.

A grammar G is ambiguous if it generates at least one string ambiguously.

Ambiguity is about grammar, not just language

The same language may have both ambiguous and unambiguous grammars.

Ambiguous Expression Grammar

Consider

$$E \to E + E \mid E * E \mid (E) \mid a.$$

The string

`a+a*a`

can be parsed as either

`a+(a*a)`

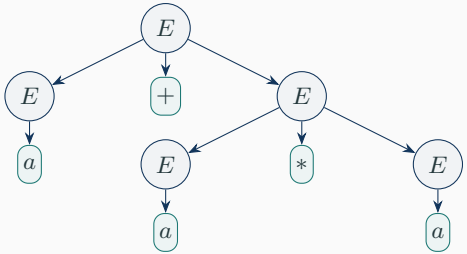
or

`(a+a)*a`.

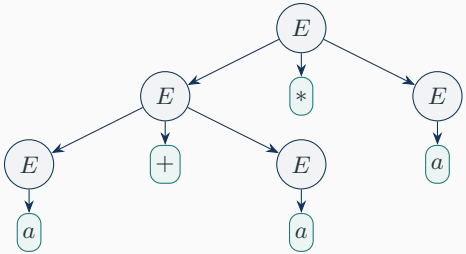
Problem

The grammar says that $+$ and $*$ are structurally equal, so it fails to encode precedence.

Two Parse Trees for $a + a * a$



$a + (a * a)$



$(a + a) * a$

Removing Expression Ambiguity

We introduce levels:

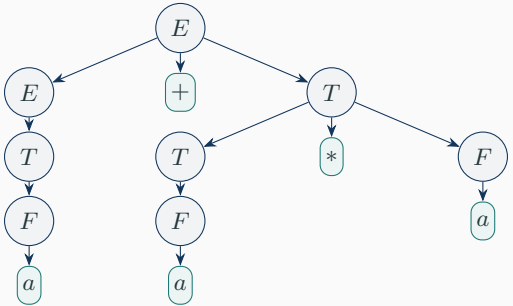
- F: factor, cannot be split by an adjacent operator;
- T: term, product of factors;
- E: expression, sum of terms.

$$E \to E + T \mid T,$$
$$T \to T * F \mid F,$$
$$F \to (E) \mid a.$$

Result

Multiplication is forced below addition in the tree, so $*$ binds more tightly than $+$.

Parse Tree After Removing Ambiguity



`a+a*a` is forced to mean `a+(a*a)`.

Ambiguity Example: Dangling Else

A naive statement grammar:

$$S \to \texttt{if}\ E\ \texttt{then}\ S \mid \texttt{if}\ E\ \texttt{then}\ S\ \texttt{else}\ S \mid \texttt{other}.$$

The string

```
if E then if E then other else other
```

can attach else to:

Inner if

```
if E then (if E then other else other)
```

Outer if

```
if E then (if E then other) else other
```

Repairing Dangling Else

Separate matched and unmatched statements.

$$
\begin{aligned}
&S \to M \mid U,\\
&M \to \texttt{other} \mid \texttt{if}\ E\ \texttt{then}\ M\ \texttt{else}\ M,\\
&U \to \texttt{if}\ E\ \texttt{then}\ S \mid \texttt{if}\ E\ \texttt{then}\ M\ \texttt{else}\ U.
\end{aligned}
$$

Convention encoded

Every `else` attaches to the nearest unmatched `if`.

Structural Ambiguity in Balanced Parentheses

The grammar

$$S \to SS \mid (S) \mid \varepsilon$$

correctly generates balanced parentheses, but it is ambiguous.

For example:

$$()\,()\,()$$

can be grouped as

$$(()\ ())\ ()$$

or as

$$()\ (()\ ()).$$

Lesson

A language may have an obvious structure, but a grammar may still give multiple parse trees for concatenation.

Inherent Ambiguity

A CFL L is inherently ambiguous if every CFG for L is ambiguous.

A standard example is:

$$L = \{a^n b^n c^m d^m \mid n, m \geq 1\} \cup \{a^n b^m c^m d^n \mid n, m \geq 1\}.$$

Why ambiguity is unavoidable

Strings of the form

$$a^n b^n c^n d^n$$

belong to both sides of the union.

Grammar for the Inherently Ambiguous Example

$$
\begin{aligned}
S &\to AB \mid C,\\
A &\to aAb \mid ab,\\
B &\to cBd \mid cd,\\
C &\to aCd \mid aDd,\\
D &\to bDc \mid bc.
\end{aligned}
$$

Branch AB

Matches a's with b's and c's with d's.

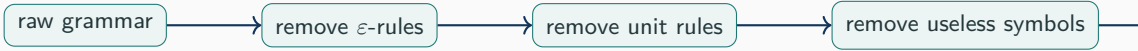
Branch C

Matches outer a, d and inner b, c.

Simplifying Grammars

Why Simplify Grammars?

Before converting to normal forms or running parsing algorithms, we clean up grammar rules.



Goal

Keep the same language while removing grammar clutter that complicates algorithms.

Useless Symbols

A symbol is useful if it is both:

1. generating: it can derive some terminal string;
2. reachable: it can appear in some derivation from the start symbol.

$$\text{useful} = \text{generating} + \text{reachable}.$$

Generating

$X \overset{*}{\Rightarrow} w$ for some $w \in \Sigma^*$.

Reachable

$S \overset{*}{\Rightarrow} \alpha X \beta$.

Worked Example: Useless Symbols

Consider

$$S \to AB \mid a,$$
$$A \to b,$$
$$B \to CD,$$
$$C \to c,$$
$$D \to DE,$$
$$E \to e,$$
$$F \to f.$$

Generating symbols

$$A, C, E, F, S$$

D is not generating, so B is not generating.

Reachable symbols

$$S, A, B, C, D, E$$

F is generating but unreachable.

Removing Useless Symbols

From the grammar

$$S \to AB \mid a, \quad A \to b, \quad B \to CD, \quad C \to c,$$
$$D \to DE, \quad E \to e, \quad F \to f,$$

remove symbols that are not both generating and reachable.

Result

$$S \to a, \qquad A \to b.$$

But after removing unreachable symbols again, only

$$S \to a$$

remains.

Reason

The branch $S \to AB$ can never terminate because B cannot generate a terminal string.

Nullable Variables and ε-Productions

A variable A is nullable if

$$A \overset{*}{\Rightarrow} \varepsilon.$$

If A is nullable and appears in a production body, then that occurrence may be kept or omitted.

Example:

$$B \to CAD$$

with A nullable gives both

$$B \to CAD \qquad \text{and} \qquad B \to CD.$$

If several nullable occurrences appear

Generate all omission patterns, then remove duplicates and forbidden empty bodies unless the start variable must keep ε.

Worked Example: Eliminating ε-Productions

Consider optional query filters:

$$S \to \texttt{select}(F, W),$$
$$W \to \texttt{where}(P) \mid \varepsilon,$$
$$F \to \texttt{name} \mid \texttt{age},$$
$$P \to F\texttt{>18}.$$

Here W is nullable.

So the rule

$$S \to \texttt{select}(F, W)$$

becomes

$$S \to \texttt{select}(F, W) \mid \texttt{select}(F,).$$

Engineering note

Real languages often design syntax to avoid ugly leftover punctuation when optional parts disappear.

A Cleaner Optional-Filter Grammar

Better grammar:

$$
\begin{aligned}
S &\to \texttt{select}(F) \mid \texttt{select}(F, W),\\
W &\to \texttt{where}(P),\\
F &\to \texttt{name} \mid \texttt{age},\\
P &\to F\texttt{>18}.
\end{aligned}
$$

Lesson

Grammar design choices affect both syntax quality and later simplification steps.

Unit Productions

A unit production has the form

$$A \to B,$$

where both A and B are variables.

Unit productions are often alias or forwarding rules:

$$E \to T, \qquad T \to F, \qquad F \to I.$$

Elimination idea

If $A \overset{*}{\Rightarrow} B$ using only unit productions, then A should inherit every nonunit production of B.

Worked Unit-Pair Closure

Consider

$$E \to T \mid E + T, \quad T \to F \mid T * F, \quad F \to I \mid (E), \quad I \to a \mid b.$$

Unit pairs include:

$$(E, E), (E, T), (E, F), (E, I),$$
$$(T, T), (T, F), (T, I),$$
$$(F, F), (F, I), (I, I).$$

Example inheritance

Because (E, I) is a unit pair and $I \to a \mid b$, add

$$E \to a \mid b.$$

After Removing Unit Productions

From the previous grammar, the nonunit productions inherited by each variable are:

$$
\begin{aligned}
E &\to E + T \mid T * F \mid (E) \mid a \mid b, \\
T &\to T * F \mid (E) \mid a \mid b, \\
F &\to (E) \mid a \mid b, \\
I &\to a \mid b.
\end{aligned}
$$

Then delete the unit rules

Rules such as $E \to T$, $T \to F$, and $F \to I$ are no longer needed.

Chomsky Normal Form

Chomsky Normal Form

A grammar is in Chomsky Normal Form if every production has one of the forms:

$$A \to BC, \qquad A \to a,$$

where A, B, C are variables and a is a terminal.

If the language contains ε, we also allow the start rule

$$S_0 \to \varepsilon,$$

provided S_0 does not appear on the right-hand side.

Theorem

Every context-free language has an equivalent grammar in Chomsky Normal Form.

Why CNF Matters

CNF is not designed for human readability.

It is designed for algorithms and proofs.

Uniform parse trees

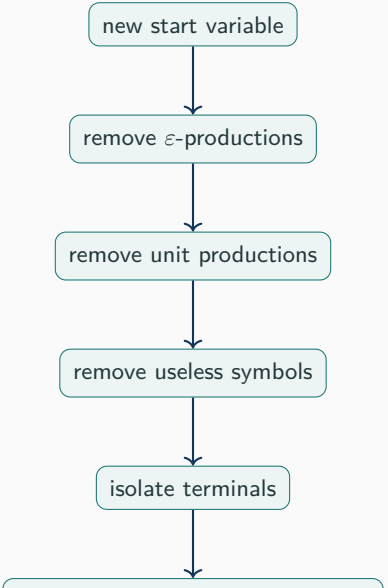
Every internal node has exactly two children, except terminal leaves.

Dynamic programming

CYK can combine two smaller substrings using one rule

$$A \to BC.$$

CNF Conversion Pipeline



CNF Worked Example: Starting Grammar

Use a small list grammar:

$$S \to [L],$$
$$L \to I \mid I, L \mid \varepsilon,$$
$$I \to a \mid b.$$

It generates strings like:

`[]`, `[a]`, `[a,b]`, `[b,a,a]`.

Problems for CNF

There is an ε-production, unit production, terminals in long bodies, and bodies longer than two.

Step 1: Add a New Start Variable

Create

$$S_0 \to S.$$

Now the grammar is:

$$\begin{aligned} S_0 &\to S, \\ S &\to [L], \\ L &\to I \mid I, L \mid \varepsilon, \\ I &\to a \mid b. \end{aligned}$$

Why?

The new start variable protects the original start symbol during later transformations, especially when ε is involved.

Step 2: Remove ε-Production

The nullable variable is L. Since

$$S \to [L]$$

contains nullable L, add the version where L is omitted:

$$S \to [].$$

Remove $L \to \varepsilon$:

$$\begin{aligned} S_0 &\to S, \\ S &\to [L] \mid [], \\ L &\to I \mid I, L, \\ I &\to a \mid b. \end{aligned}$$

The language still contains []

It is now generated by $S \to []$, not by $L \to \varepsilon$.

Step 3: Remove Unit Productions

Unit productions:

$$S_0 \to S, \qquad L \to I.$$

Inherit nonunit productions:

$$\begin{aligned} S_0 &\to [L] \mid [], \\ S &\to [L] \mid [], \\ L &\to I, L \mid a \mid b, \\ I &\to a \mid b. \end{aligned}$$

Now remove useless symbols if desired. Since Sot reachable from S_0 anymore, remove S:

$$\begin{aligned} S_0 &\to [L] \mid [], \\ L &\to I, L \mid a \mid b, \\ I &\to a \mid b. \end{aligned}$$

Step 4: Isolate Terminals in Long Bodies

Introduce variables for terminals that appear in longer bodies:

$$A \to a, \quad B \to b, \quad LB \to [, \quad RB \to], \quad C \to , .$$

Replace terminals inside longer bodies:

$$\begin{aligned} S_0 &\to LB\ L\ RB \mid LB\ RB, \\ L &\to I\ C\ L \mid a \mid b, \\ I &\to a \mid b, \\ LB &\to [, \qquad RB \to], \qquad C \to , . \end{aligned}$$

Single-terminal rules are already CNF

Rules like $L \to a$ and $I \to b$ are allowed.

Step 5: Break Long Bodies into Binary Rules

Break

$$S_0 \to LB\ L\ RB$$

using

$$X_1 \to L\ RB, \qquad S_0 \to LB\ X_1.$$

Break

$$L \to I\ C\ L$$

using

$$X_2 \to C\ L, \qquad L \to I\ X_2.$$

Final CNF-style grammar:

$$\begin{aligned} S_0 &\to LB\ X_1 \mid LB\ RB, \\ X_1 &\to L\ RB, \\ L &\to I\ X_2 \mid a \mid b, \\ X_2 &\to C\ L, \\ I &\to a \mid b, \\ LB &\to [, \quad RB \to], \quad C \to , . \end{aligned}$$

CYK Membership Testing

CYK: Membership Testing for CFLs

Given:

- a grammar G in CNF;
- an input string $w = a_1 a_2 \cdots a_n$.

CYK builds a triangular dynamic-programming table.

Entry X_{ij} contains all variables A such that

$$A \overset{*}{\Rightarrow} a_i a_{i+1} \cdots a_j.$$

Acceptance criterion

$$w \in L(G) \quad \Longleftrightarrow \quad S \in X_{1n}.$$

CYK Recurrence

Base case:

$$A \in X_{ii} \quad \text{if} \quad A \to a_i.$$

For $i < j$, include $A \in X_{ij}$ if there exists k with

$$i \le k < j,$$

and variables B, C such that

$$B \in X_{ik}, \qquad C \in X_{k+1,j}, \qquad A \to BC.$$

Meaning

Try every split of the substring, and see whether a CNF rule combines the two halves.

CYK Worked Example: Grammar

Use the CNF grammar for $\{a^n b^n \mid n \geq 1\}$:

$$S \to AB \mid AT,$$
$$T \to SB,$$
$$A \to a,$$
$$B \to b.$$

Test the string

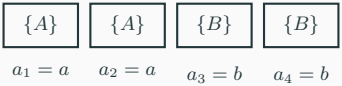
$$w = \texttt{aabb}.$$

Goal

We will determine whether $S \in X_{14}$.

CYK Table: Base Row

$$w = a_1a_2a_3a_4 = \texttt{aabb}.$$



Reason

$$A \to a, \qquad B \to b.$$

So each one-letter cell receives the variable that generates that terminal.

CYK Table: Length 2 Substrings

Substrings of length 2:

$$\texttt{aa}, \quad \texttt{ab}, \quad \texttt{bb}.$$

Only ab can be generated by S, because

$$S \to AB, \quad A \in X_{22}, \quad B \in X_{33}.$$

$\emptyset$	$\{S\}$	$\emptyset$	
$\{A\}$	$\{A\}$	$\{B\}$	$\{B\}$
a	a	b	b

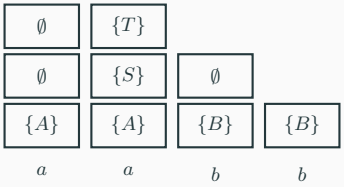
CYK Table: Length 3 Substrings

For substring abb, we have:

$$X_{23} = \{S\}, \qquad X_{44} = \{B\}, \qquad T \to SB.$$

Therefore

$$T \in X_{24}.$$



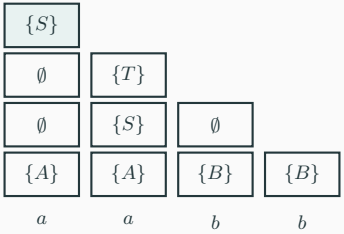
CYK Table: Final Cell

For the full string aabb:

$$X_{11} = \{A\}, \qquad X_{24} = \{T\}, \qquad S \to AT.$$

Therefore

$$S \in X_{14}.$$



Since $S \in X_{14}$, CYK accepts aabb.

Why CYK Runs in $O(n^3)$

There are $O(n^2)$ table cells.

For each cell X_{ij}, we try $O(n)$ split positions:

$$i \leq k < j.$$

For each split, we check whether some rule

$$A \to BC$$

combines variables from the left and right cells.

Main bound

With the grammar fixed, the running time is

$$O(n^3).$$

Recap and Bridge to PDA

Today's Big Picture

CFGs describe recursive construction.

Rules

A finite set of productions expands variables into structure.

Trees

Parse trees reveal the hierarchy hidden inside the string.

Algorithms

CNF and CYK turn grammar membership into dynamic programming.

Practice Problems

Design CFGs for the following languages.

1. $\{a^n b^m c^n \mid n, m \geq 0\}$
2. $\{w\#x \mid w, x \in \{0,1\}^*,\ |w| = |x|\}$
3. Properly nested XML-like tags using only <a> and .
4. Boolean formulas over p, q using $\neg, \wedge, \vee$, with standard precedence.
5. $\{w \in \{0,1\}^* \mid w$ has equal numbers of 01 and $10\}$.

Challenge

For each grammar, state why invalid strings cannot be generated.

Bridge to Pushdown Automata

A CFG tells us how a string may be generated.

The next question is:

> Can a machine recognize exactly the strings generated by a CFG?

The answer is yes.

Next model

A pushdown automaton is a finite automaton equipped with one unbounded stack.

The stack will remember exactly the kind of nested and matched structure that CFGs generate.